

ChE/EID 488: Convex Optimization Techniques
Professor Davis

Optimizing Darkovian Subrace Damage in AdventureQuest

Andrew Yuan

May 2025

Abstract

This project investigates how to optimally assign stat points in the online role-playing game (RPG) *AdventureQuest* to maximize single-turn damage for each of the Darkovian subraces: Vampire, Werewolf, and Werepyre. In the game, players are given 750 points to allocate among six core stats: Strength (STR), Dexterity (DEX), Intellect (INT), Endurance (END), Charisma (CHA), and Luck (LUK). Each stat contributes differently to combat effectiveness depending on the character's subrace and the type of attack used, e.g. Melee, Ranged, Magic, or Spell. The main objective of this project was to determine the best possible stat point allocation for each subrace and attack type to deal the highest damage possible during realistic gameplay.

We modeled this problem first as an Integer Linear Program (ILP), using simplified stat-based damage formulas, in-game subrace ability bonuses, and attack type multipliers only. The problem was then relaxed into a Linear Program (LP) to facilitate analysis and application of first and second-order necessary conditions. Interestingly, the relaxed solutions laid at integer multiples of 5 — the in-game fixed increments used for stat training — also solving the ILP. Later, the model was expanded by introducing pet and guest multipliers that nonlinearly amplify damage, resulting in a Quadratic Integer Program (QIP). This extension better reflects in-game stat interactions and incorporates realistic support units like Poelala and Angra Linnorm. Once again, the relaxed Quadratic Program (QP) solutions aligned with our QIP.

For each subrace and attack type combination, we treated them as an individual subproblem and built separate Pyomo models in Python; the LPs were solved using GLPK and the QPs were solved using Gurobi. We found that damage outcomes changed significantly after including subrace-specific ability multipliers and support unit effects. Vampires, for instance, emerged as optimal for Spell attacks with damage exceeding 280, while Werewolves dominated physical Melee builds with 73 damage. Werepyres, which were initially underperforming without multipliers, became comparable in Melee damage and exceptional in Magic setups, due to their hybrid scaling on both STR and INT. These results highlight how accounting for additional mechanics such as in-game buffs and nonlinear stat bonuses can reveal viable strategies that a simple linear model would overlook.

This project demonstrates the power of mathematical optimization to model complex decision-making in RPGs. In fact, this framework can be readily applied to validate or challenge prevailing “meta” strategies by identifying optimal builds under precise constraints. Dual variable analysis also provides insight into which stats constrain performance the most, which can guide newer players in prioritizing stat allocation order. Future work could extend the model to include other gameplay features such as: style bonuses, item effects, and level-scaling mechanics, or multi-turn combat, accounting for resource usage like skill points (SP) and mana points (MP).

This framework serves as a flexible foundation for analyzing and optimizing strategic choices in AdventureQuest and potentially other turn-based RPGs.

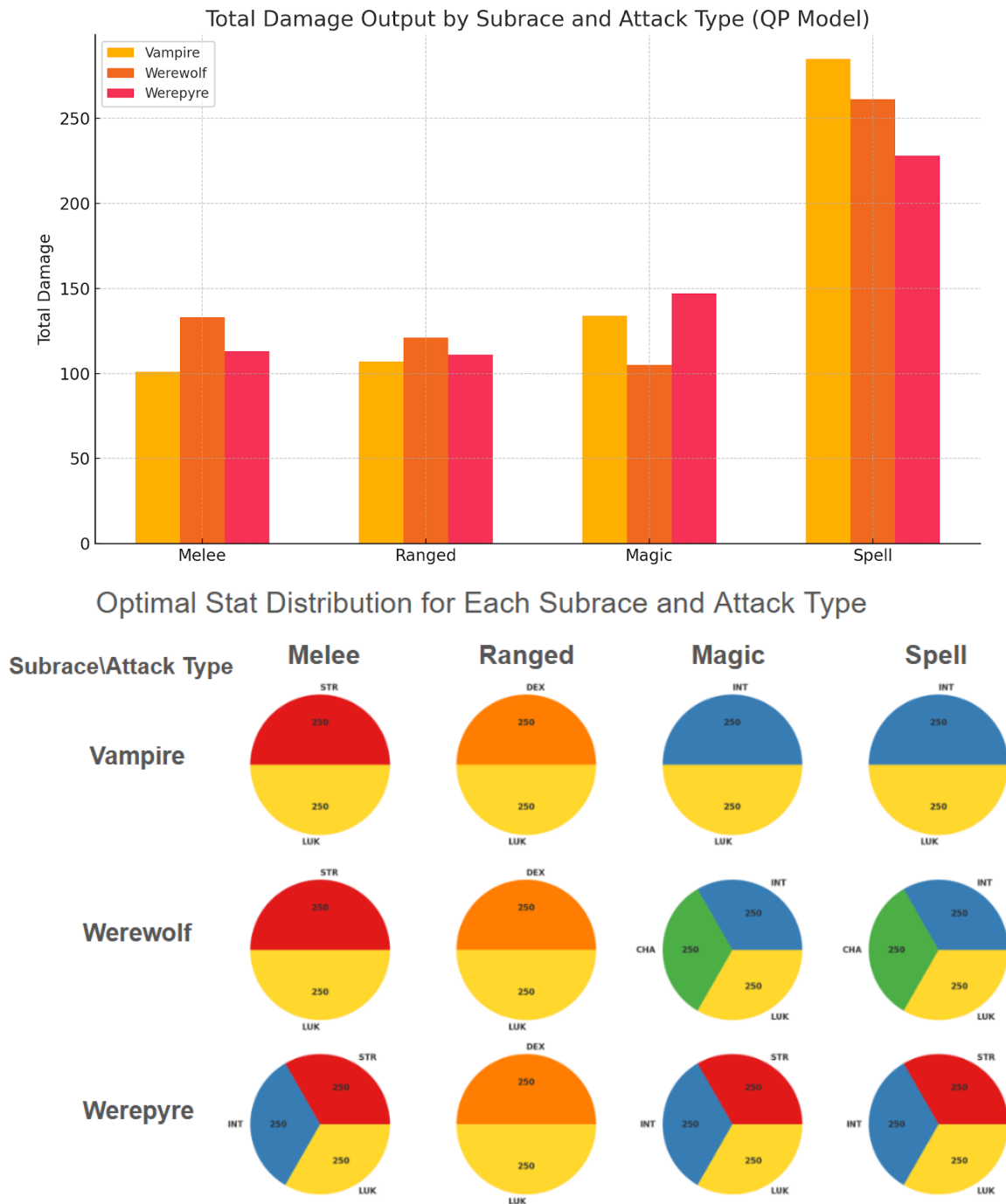


Figure: On the top, total damage output by subrace and attack type under the full QP model with pet and guest buffs. Each bar represents the final optimized damage after applying stat-based multipliers, ability effects, and type-of-attack bonuses for the Vampire, Werewolf, and Werepyre subraces. On the bottom, Each pie chart represents the optimal distribution of stats for each Subrace and attack type.

Introduction

AdventureQuest (AQ) is a long-standing [1] online single player role-playing game (RPG) where players can create and customize their character to explore in the in-game universe of *Lore*. Launched in 2002, AQ features turn-based combat against monsters that appear as the player adventures. As players defeat monsters, they earn both experience points and gold, which can be used to level up the character and purchase new and stronger equipment. AQ has evolved over the course of two decades, but these core mechanics have remained the same.



Figure 1: Official Adventure Quest Game Logo along with many notable characters of the game.

The primary objective of AQ is to build a powerful character capable of defeating increasingly stronger enemies and completing quests to advance the storyline and unlock rare rewards. As a player levels up, they gain stat points which can be allocated to various character attributes such as Strength (STR), Dexterity (DEX), Intellect (INT), Endurance (END), Charisma (CHA), and Luck (LUK). Each of these attributes influences combat performance in different ways:

STR — Shorthand for Strength. This increases the damage and accuracy you deal with Melee weapons.

DEX — Shorthand for Dexterity. This increases the damage and accuracy of Ranged weapons, and the accuracy of pets and guests.

INT — Shorthand for Intelligence. This increases the damage and accuracy of Magic weapons and spells.

END — Shorthand for Endurance. This increases the amount of health the player has, doubling the HP pool at max level at 250 points.

CHA — Shorthand for Charisma. This increases the damage and accuracy of your pets and guests.

LUK — Shorthand for Luck. This increases your damage in the form of Lucky Strikes, meaning 10% of your attacks deal additional damage based on the amount of LUK you have. LUK also increases your accuracy and increases the chance of inflicting status effects.

As monsters become tougher to battle, proper stat allocation to maximize combat effectiveness becomes a core feature of interest in character development. Thus the interest of this project is to discover the optimal stat distribution to maximize damage output.



Figure 2: Images of the stat training menu where stat points can be allocated/reallocated. In the left image, there are 5 available stat points to allocate, and in the right image, we can see they are allocated to STR. Points can be incremented or decremented in multiples of 5.

In 2018, the core gameplay of AQ was revolutionized with the introduction of Darkovian Subraces. The three main subraces — Vampire, Werewolf, and Werepyre (a hybrid of the two) — are unique character transformations that offer distinct combat styles and stat scaling. Each subrace specializes in different damage types and is granted access to powerful abilities at higher levels which can amplify damage in unique ways.



Figure 3: (left to right) Vampire, Werewolf, and Werepyre subrace base forms.

The initial focus will be on maximizing our one-turn damage output without factoring in any bonus stats from pets or guests. In this case, only stat allocation and subrace perks determine the one-turn damage output.

Before we delve into the problem formulation, some major assumptions need to be clarified:

- 1) Characters must be at least level 40 in order to first select a Darkovian subrace. In this problem, characters are assumed to be at max level (150) with all 750 available stat points as this offers the most flexibility in choosing where to allocate stat points. Attaining the maximum level requires significant playtime; while many players do not achieve this, the problem is still relevant to characters at lower levels who just have less available stat points.
- 2) Each character can only be one subrace.
- 3) All subraces are assumed to use the same weapon type with identical base damage, i.e. weapon/spell base damage will be the same across all subraces and attack types. In AQ, weapon base damage is described relative to melee damage; magic weapons and spells have base multipliers of 0.75 times and 2 times melee damage, respectively.
- 4) Any attack can only be of one type (i.e. melee, range, magic, or spell).



Figure 4: An image of Zealot's Wrath, one of the few weapons in-game where attack type can be toggled. Zealot's Wrath can switch the attack type by clicking on the weapon itself in-game.

- 5) Subraces are assumed to be at the highest class level (10) with the final transformation active. We will also assume each Subrace gives a 1.0x Armor lean, meaning that we do not have additional damage multipliers based on the Subrace transformation excluding abilities, see Figure 6.
- 6) Abilities and attacks may consist of multiple hits in one turn (e.g., 3 to 4 per use), but all subraces are assumed to have the same number of hits for their respective basic and spellcasting attacks. We count multiple hits as one attack and just add up the total damage, this way, the total damage dealt in a single turn will be the focus of the optimization.
- 7) Lucky Strikes will be accounted for as a bonus to base damage proportional to their chance of occurrence. Certain buffs only apply to non-Lucky Strike damage but this model will apply them uniformly to all attacks to simplify the computations.
- 8) Enemy resistances are assumed to be 100% and uniform across all the subraces, meaning the player does standard damage and resistances can be ignored during the optimization.
- 9) In-game items may provide buffs/benefits to the player: we will assume the player does not equip any such items.
- 10) Lastly, the model assumes the player always uses their most optimal damage-boosting abilities without any cooldown or resource restrictions (e.g., skill points/mana points).

In-game, the stats can be incremented in multiples of 5 points, capping out at 250 points for any individual stat.

Fako Mustacho			
Race	Human Werewolf	78228739	
Class	Fighter		
Clan	Glacius		
Information		Combat Defense	
Level	150	Melee	58
Gold	376171304	Ranged	58
Tokens	116	Magic	58
Attributes		Element Modifier	
Strength	250	Fire	87 %
Dexterity	200	Water	20 %
Intellect	0	Wind	73 %
Endurance	50	Ice	87 %
Charisma	0	Earth	45 %
Luck	250	Energy	87 %
		Light	45 %
		Darkness	45 %
Equipment			
	Shinyaro Form		
	Kindred Spirits' Enlightenm		x 16
	Kindred Spirits' Devotion		x 26
	None		

Figure 5: Character Fako Mustacho with stats allocated to STR, DEX, END, and LUK. In this image, Fako Mustacho is a Werewolf subrace.

Since Adventure Quest does not provide an official damage formula for subrace bonuses, with our assumption and the normalization of equipment across all subraces, a simplified approximation is used based on community consensus and in-game observation:

$$\text{Damage} = (\text{Ability Multiplier}) \cdot (\text{Core-Stat damage}) \cdot (\text{Type of Attack Bonus})$$

In order to calculate the maximum damage output, various stats and their effects need to be considered. From these Stat Damage formulas taken from the AQ Wiki [2]:

```

Melee Weapon: STR/8
Ranged Weapon: DEX/8
Magic Weapon: INT*3/32
Melee Skill/Spell: STR/4
Ranged Skill/Spell: DEX/4
Magic Skill/Spell: INT/4
Pets & Guests: CHA/15

```

Each hit of all player attacks (Weapons, Skills, and Spells) has a 10% chance of adding damage in the form of a Lucky Strike, which increases core stat damage by $\text{LUK} \cdot 3/8$ for that hit.

- On average, this bonus results in an increase of $\text{LUK} \cdot 3/80$.

Figure 6: Outlining Core-Stat Damage Formulas [2]

Thus including lucky strike damage, the damage output from a normal attack calculated from the stat distribution is given as:

```

Melee Weapon: STR / 8 + LUK * 3 / 80
Ranged Weapon: DEX / 8 + LUK * 3 / 80
Magic Weapon: INT * 3 / 32 + LUK * 3 / 80
Melee Skill/Spell: STR / 4 + LUK * 3 / 80
Ranged Skill/Spell: DEX / 4 + LUK * 3 / 80
Magic Skill/Spell: INT / 4 + LUK * 3 / 80
Pets: CHA / 15 + LUK / 50
Guests: CHA / 15

```

Figure 7: Outlining Average Core-Stat Damage formulas with Lucky Strikes included [2]

STR, DEX, and INT impact damage output the most and thus are often referred to as “Main stats,” and CHA, END, LUK are referred to as “support stats.” In this scenario, since battles are “one turn” long, support stats such as Endurance are not optimal to invest in as survivability is not a relevant factor. This is not always representative of the actual game; certain monsters with high health points or high damage dealing capabilities necessitate higher survivability, so this will be expanded upon in the extension.

Since each subrace is modeled at max class level, all abilities are assumed to be unlocked. For instance, the Werewolf subrace’s level 3 ability *Lycan Reflexes* provides a +50 DEX stat increase and the level 8 ability *Lycan Strength* provides a +50 STR stat increase. The level 10 ability *Become the Beast* provides bonus damage and utilizes a different damage formula. From the BattleOn forums, the description reads, “The Bonus Stat damage is increased to STR/4 for Melee, STR/5 + DEX/10 for Ranged, and INT/4 for Magic” for Werewolves [4].



Figure 6: The Werewolf subrace abilities. The level 10 ability *Become the Beast* is highlighted and the ability description is displayed.

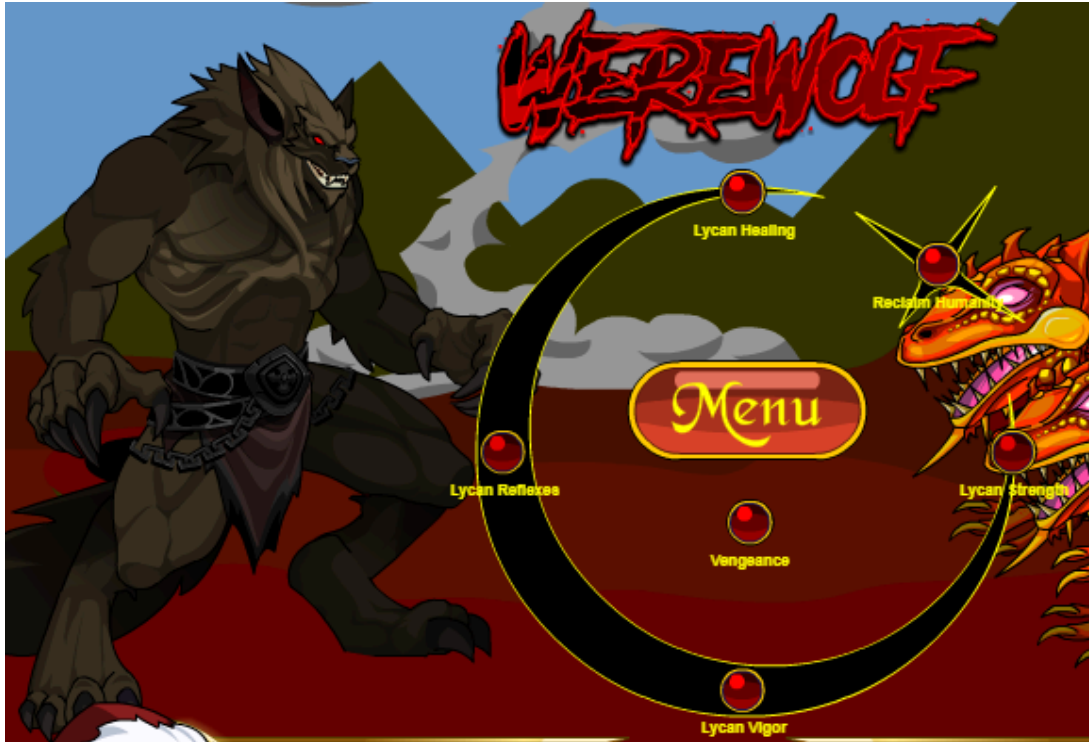


Figure 7: After toggling *Become the Beast*, the player assumes a different form now only with a subset of the abilities available pre-transformation.

Similarly, the level 10 class ability *Evolve* for Vampires reads, “Bonus Stat damage is increased to STR/4 for Melee, STR/5 + DEX/10 for Ranged, and INT/4 for Magic” [5], and the level 10 class ability for Werepyre *Become the Beast*, “Bring all your weapon attacks up to skill level and force them to use ‘hybrid’ stats. For spells, it causes them to deal +50% damage and forces them to use ‘hybrid’ stats”. These “hybrid” stats reference an alternate damage formula for hybrids [6]:

Melee: $STR/16 + INT/16$

Magic Weapon and Spells: $3 \cdot STR/40 + 3 \cdot INT/40 + LUK/40$

Factoring in Lucky strikes, this comes out to:

$STR/16 + INT/16 + 3 \cdot LUK/80$ and $3 \cdot STR/40 + 3 \cdot INT/40 + 5 \cdot LUK/80$

Note that Ranged and Spell-based attacks still utilize the same formula as the other subraces.



Figure 8: (Left to right) The final transformations of Vampire, Werewolf, and Werepyre via the level 10 class abilities Evolve and Become the Beast

Lastly, as mentioned beforehand, each normal attack can be of one type: melee, range, magic, or spell. Magic weapons are inherently stronger than melee weapons due to certain in-game mechanics such as weapon specials (this is ignored since we assume normal attacks only) thus a 0.75 debuff is used in-game for balancing purposes. Spells by design are meant to “nuke” the enemy thus are twice as effective as melee weapons [2].

However, these buffs are all constant multipliers or flat additions to the effective stat values used in the damage formulas. Since they only shift/scale the feasible region, they do not influence the optimization. In other words, the presence of a constant multiplier or additive buff simply scales the final damage output but does not change which stat allocation yields the maximum value. Therefore, these constant buffs can be factored out from the objective function to solely focus on the underlying stat-based damage terms. The optimal value can then be rescaled to include these buffs and find the maximum damage output.

With these assumptions in place, our optimization problem in words is:

Maximize: The basic attack damage from one Darkovian subrace in one turn

Subject to:

- 1) Total stat point allocation cannot be below 0 or exceed 750.
- 2) Point allocation towards each stat cannot be below 0 or exceed 250.
- 3) Stat point allocations can only be multiples of 5.

Problem Description

Using the core-stat damage formulas listed on the AQ Wiki and bonus damage formulas for each subrace, the objective functions for each subrace and attack type can be constructed. Note that the *quest* to find the maximum damage output across all the Darkovian subraces across all attack types can be divided into subproblems. Each subproblem consists of one Darkovian subrace along with one attack type. The general objective function in scalar form for each subproblem is as follows:

$$\begin{aligned} \max_{\substack{x_i \in \mathbb{Z} \\ x_i \in \{0,5,\dots,250\}}} f(\mathbf{x}) &= \sum_{i=1}^6 c_i x_i \\ \text{subject to } \sum_{i=1}^6 x_i &\leq 750 \\ 0 &\leq x_i \leq 250 \end{aligned}$$

We define $\mathbf{x}$ to be such that: $x_1 = \text{STR}$, $x_2 = \text{DEX}$, $x_3 = \text{INT}$, $x_4 = \text{END}$, $x_5 = \text{CHA}$, and $x_6 = \text{LUK}$. The decision variable x_i where $i = 1 \dots 6$ represents the distribution of stats for the player and these stat points can only be multiples of 5 up to 250. Here c_i represents the weighted coefficients that determine how each stat contributes to the damage output for a given attack type.

Now we construct the problem in matrix/vector form to contain all the coefficients and different subcases in a clean manner. To transform a maximization problem into an equivalent minimization, we simply multiply the objective function by negative 1:

$$\begin{aligned} \min_{\substack{x_i \in \mathbb{Z} \\ x_i \in \{0,5,\dots,250\}}} \mathbf{c}^T \mathbf{x} \\ \text{subject to } \begin{bmatrix} \mathbf{1}_{1 \times 6} \\ -I_{6 \times 6} \\ I_{6 \times 6} \end{bmatrix} \mathbf{x} &\leq \begin{bmatrix} 750 \\ \mathbf{0}_{6 \times 1} \\ 250 \cdot \mathbf{1}_{6 \times 1} \end{bmatrix} \\ \text{where } A &= \begin{bmatrix} \mathbf{1}_{1 \times 6} \\ -I_{6 \times 6} \\ I_{6 \times 6} \end{bmatrix}, \quad b = \begin{bmatrix} 750 \\ \mathbf{0}_{6 \times 1} \\ 250 \cdot \mathbf{1}_{6 \times 1} \end{bmatrix} \end{aligned}$$

With these considerations, we can construct the $\mathbf{c}$ vector containing the coefficients for our objective function displayed in the table shown below, noting that $\mathbf{c}$ is scaled by negative 1:

Subrace	Melee	Ranged	Magic	Spell
Vampire	$[\frac{-1}{4}, 0, 0, 0, 0, \frac{-3}{80}]$	$[\frac{-1}{5}, \frac{-1}{10}, 0, 0, 0, \frac{-3}{80}]$	$[0, 0, \frac{-1}{4}, 0, 0, \frac{-3}{80}]$	$[0, 0, \frac{-1}{4}, 0, 0, \frac{-3}{80}]$
Werewolf	$[\frac{-1}{4}, 0, 0, 0, 0, \frac{-3}{80}]$	$[\frac{-1}{5}, \frac{-1}{20}, 0, 0, 0, \frac{-3}{80}]$	$[0, 0, \frac{-1}{4}, 0, 0, \frac{-3}{80}]$	$[0, 0, \frac{-1}{4}, 0, 0, \frac{-3}{80}]$
Werepyre	$[\frac{-1}{16}, 0, \frac{-1}{16}, 0, 0, \frac{-3}{80}]$	$[0, \frac{-1}{8}, 0, 0, 0, \frac{-3}{80}]$	$[\frac{-3}{40}, 0, \frac{-3}{40}, 0, 0, \frac{-5}{80}]$	$[\frac{-3}{40}, 0, \frac{-3}{40}, 0, 0, \frac{-5}{80}]$

Table 1: Stat weighting vectors c for different subrace and attack types derived from the core-stat damage formulas

The objective function is linear, defined as a weighted sum of the stat variables. The coefficient multipliers do not weigh the stats and are only flat bonuses. Additionally, all constraints are also linear, including the total stat budget constraint and variable bounds. Since the decision variables are discrete, (integer multiples of 5), this makes the problem the integer type thus this is an Integer Linear Problem (ILP).

The feasible region Ω is the set of all 6-dimensional integer vectors x representing valid stat allocations, where each stat can take values from 0 to 250 in increments of 5, and the total stat budget does not exceed 750 points. The vector $x = 0 \in \Omega$, since it trivially satisfies all constraints. Thus, $\Omega \neq \emptyset$, and the problem is feasible. Moreover, each $x \in \Omega$ has norm

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_6^2} = \sqrt{6 \cdot 250^2} < 800, \text{ therefore } \Omega \text{ is contained within a}$$

Euclidean ball of radius 800 centered at the origin. Thus Ω is bounded. Thus we guarantee the objective function has a minimum on the feasible set. Because the feasible region is not continuous, both First-Order Necessary Conditions (FONC) and Second-Order Necessary Conditions (SONC) are not applicable in this problem. These conditions require continuous domains, which do not hold in this integer framework.

Instead, we consider the Linear Programming (LP) relaxation of the problem by disregarding the integer multiples of 5 criteria:

$$\begin{aligned}
 & \min_{x_i \in \mathbb{R}} \quad \mathbf{c}^T \mathbf{x} \\
 & \text{subject to} \quad \begin{bmatrix} \mathbf{1}_{1 \times 6} \\ -I_{6 \times 6} \\ I_{6 \times 6} \end{bmatrix} \mathbf{x} \leq \begin{bmatrix} 750 \\ \mathbf{0}_{6 \times 1} \\ 250 \cdot \mathbf{1}_{6 \times 1} \end{bmatrix} \\
 & \text{where } A = \begin{bmatrix} \mathbf{1}_{1 \times 6} \\ -I_{6 \times 6} \\ I_{6 \times 6} \end{bmatrix}, \quad b = \begin{bmatrix} 750 \\ \mathbf{0}_{6 \times 1} \\ 250 \cdot \mathbf{1}_{6 \times 1} \end{bmatrix}
 \end{aligned}$$

Here the relaxed feasible set is a compact polytope in $\mathbb{R}^6$, and since the objective function is linear and thus continuous, we can apply Weierstrass's Extreme Value Theorem. This theorem guarantees that our objective function attains both a maximum and a minimum on the feasible set. Furthermore, because this is an LP over a polytope, the minimum is attained at a point on the boundary of the feasible region. That is, there exists an $\mathbf{x}^* \in \Omega$ such that:

$$f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad \text{for all } \mathbf{x} \in \Omega$$

hence the infimum is attained and the problem has an optimal solution.

Moreover, FONC and SONC are applicable. In fact, our objective function is linear so the Hessian of $f(\mathbf{x})$ is 0:

$$\nabla^2 f(\mathbf{x}) = \mathbf{0}_{6 \times 6}$$

This means that the SONC are automatically satisfied. Since the objective function is linear, its Hessian is zero and thus trivially positive semidefinite. Additionally, a linear objective function is convex so we now verify if the feasible set is convex. The feasible region is defined by a set of linear inequalities- each of which define a half space. Since our feasible region is the intersection of all these half-spaces, which are individually convex, our feasible region is therefore convex as well. Because both the objective function and the feasible set are convex, any point $\mathbf{x}^*$ that satisfies the FONC is guaranteed to be a global minimum though not necessarily unique since strict convexity is not satisfied (because the Hessian of our objective function is 0).

Some common methods to solve ILPs include Branch and Bound, Cutting Planes, and the hybrid method Branch and Cut. In Branch and Bound, the feasible region is recursively partitioned into subproblems (branching), and a LP relaxation is solved at each node to compute bounds [8]. These LPs are typically solved using either the Simplex Method, which moves along the edges of the feasible region to reach an optimal vertex, or Interior-Point Methods, which progress through the interior of the polytope and are often preferred for large-scale or sparse problems due to their polynomial-time convergence properties [9]. If the LP relaxation yields an infeasible solution or one worse than the current best integer solution, the corresponding branch is pruned, significantly reducing the search space. Cutting Plane methods utilize a different approach by solving the LP relaxation and then iteratively adding valid inequalities, or cuts, that eliminate fractional solutions without removing any feasible integer points. These cuts progressively tighten the feasible region of the relaxation, which converge to the integer constraints. The Branch and Cut

framework combines the strengths of both approaches. It uses branching to divide the problem space and cutting planes to improve the quality of the LP bounds at each node. This hybrid method is among the most powerful techniques used in modern ILP solvers, enabling convergence to the globally optimal integer solution in efficient time [10].

Results

The decision variables in our problem represents how many stat points to assign to each of the six core stats: Strength (STR), Dexterity (DEX), Intellect (INT), Endurance (END), Charisma (CHA), and Luck (LUK). Each variable can take values of multiple 5 from 0 to 250, and the sum of all allocated points must not exceed 750. The objective is to maximize the expected base damage output of a selected subrace and attack type (e.g., Vampire Spell or Werepyre Melee), given its scaling mechanics and multipliers. For each subrace and attack type combination, we defined a separate Pyomo model that included the appropriate damage formula and constraints. Each model included 1 total stat cap constraint, 6 bound constraints, and 6 integer constraints enforcing that each stat be a multiple of 5 which totaled to 13 constraints per instance. The solver was then run for each scenario, yielding the optimal stat distribution and the corresponding maximum damage. Because the feasible space is relatively small (due to the 5-point increments and capped total of 750 points), on average, GLPK was able to find the optimal solutions in 0.004 seconds for each case.

Below is a table of the results and the optimal stat distributions for each subrace and attack type. Note that the Vampire, Werewolf, and Werepyre *Range* do not exhaust the total available skill points. This means that regardless of where the remaining stat points are allocated to, the base damage from normal attacks do not change.

Subrace	Type	STR	DEX	INT	END	CHA	LUK	Damage (v^*)	Not Assigned N/A
Vampire	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Vampire	Range	0	250	0	0	0	250	84	STR, INT, END, CHA
Vampire	Magic	0	0	250	0	0	250	72	STR, DEX, END, CHA
Vampire	Spell	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werewolf	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Werewolf	Range	0	250	0	0	0	250	72	STR, INT, END, CHA
Werewolf	Magic	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werewolf	Spell	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werepyre	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Werepyre	Range	0	250	0	0	0	250	41	STR, INT, END, CHA
Werepyre	Magic	250	0	250	0	0	250	53	DEX, END, CHA
Werepyre	Spell	250	0	250	0	0	250	53	DEX, END, CHA

Table 2: Optimal stat distributions and resulting damage values before multipliers for ILP problem for each subrace across four attack types.

Subrace	Type	STR	DEX	INT	END	CHA	LUK	Damage (v^*)	Not Assigned N/A
Vampire	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Vampire	Range	0	250	0	0	0	250	84	STR, INT, END, CHA
Vampire	Magic	0	0	250	0	0	250	72	STR, DEX, END, CHA
Vampire	Spell	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werewolf	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Werewolf	Range	0	250	0	0	0	250	72	STR, INT, END, CHA
Werewolf	Magic	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werewolf	Spell	0	0	250	0	0	250	72	STR, DEX, END, CHA
Werepyre	Melee	250	0	0	0	0	250	72	DEX, INT, END, CHA
Werepyre	Range	0	250	0	0	0	250	41	STR, INT, END, CHA
Werepyre	Magic	250	0	250	0	0	250	53	DEX, END, CHA
Werepyre	Spell	250	0	250	0	0	250	53	DEX, END, CHA

Table 3: Optimal stat distributions and resulting damage values before multipliers for LP relaxation problem for each subrace across four attack types.

The results show that for every subrace and attack type, the optimal stat allocations produced by the ILP and its LP relaxation are identical. This means the LP relaxation yields solutions where all variables naturally fall on integer multiples of 5, satisfying the integer constraints without needing explicit enforcement. In practice, this implies the LP’s continuous optimum lies at a vertex that also happens to be an integer-feasible point. This is an important and useful insight that significantly simplifies computation since solving the LP relaxation is typically much faster and more scalable than solving the ILP, especially in large problems [9]. The agreement between the two also suggests that, for this simplified version of the problem, integer rounding or branch-and-bound techniques do not need to be invoked.

From Table 2 and 3, we observe that the maximum damage for each subrace and attack type is achieved when specific combinations of stat points are allocated. For instance, Vampire Spell reaches a peak damage of 72 when 250 points are allocated to both INT and LUK. Since each stat is trained in increments of 5, and players receive 5 stat points per level, this configuration requires a total of 500 points and is therefore achievable by level 100. Extending this logic to other subclasses, we note that the highest damage values generally correspond to total stat allocations of either 500 or 750 points. For instance, Werepyre Melee reaches its peak damage of 72 when 250 points are allocated to STR, INT, and LUK. This configuration reaches its full potential only at level 150, the maximum level cap under our model. This analysis highlights a relationship between progression and payoff; fully optimizing the Werepyre subrace requires reaching a much later stage in the game compared to other subrace-attack combinations, due to its heavier stat investment demands

Subrace	Attack	Stats Used	Total Points	Min. Level
Vampire	Melee	STR, LUK	500	100
Vampire	Ranged	DEX, LUK	500	100
Vampire	Magic	INT, LUK	500	100
Vampire	Spell	INT, LUK	500	100
Werewolf	Melee	STR, LUK	500	100
Werewolf	Ranged	DEX, LUK	500	100
Werewolf	Magic	INT, CHA, LUK	750	150
Werewolf	Spell	INT, CHA, LUK	750	150
Werepyre	Melee	STR, LUK	500	100
Werepyre	Ranged	DEX, LUK	500	100
Werepyre	Magic	STR, INT, LUK	750	150
Werepyre	Spell	STR, INT, LUK	750	150

Table 4: Optimal stats and minimum character level required to reach peak damage for each subrace and attack type based on total stat investment.

With the agreement between the ILP and its LP Relaxation, we can now include the Ability Multipliers from each Darkovian Class abilities as well as the Type of Attack (ToA) bonus to find the maximum damage output in one turn using the previous damage output formula:

Subrace	Type	Ability Multipliers	Base Damage	ToA Bonus	Damage Output
Vampire	Melee	$(0.875)(0.875)$	$\left(\frac{x_1}{4} + \frac{3x_6}{80}\right)$	1	55
Vampire	Ranged	$(0.875)(0.875)$	$\left(\frac{x_1}{5} + \frac{x_2}{10} + \frac{3x_6}{80}\right)$	1	55
Vampire	Magic	$(0.75)(0.875)$	$\left(\frac{x_3 + 50}{4} + \frac{3x_6}{80}\right)$	0.75	42
Vampire	Spell	$(1)(0.875) + 0.375$	$\left(\frac{x_3 + 50}{4} + \frac{3x_6}{80}\right)$	2	211
Werewolf	Melee	0.875	$\left(\frac{x_1 + 50}{4} + \frac{3x_6}{80}\right)$	1	65
Werewolf	Ranged	0.875	$\left(\frac{x_1 + 50}{4} + \frac{x_2 + 50}{20} + \frac{3x_6}{80}\right)$	1	65
Werewolf	Magic	0.75	$\left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$	0.75	35
Werewolf	Spell	$1 + 0.375$	$\left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$	2	198
Werepyre	Melee	1	$\left(\frac{x_1 + 35}{16} + \frac{x_3 + 35}{16} + \frac{3x_6}{80}\right)$	1	45
Werepyre	Ranged	1	$\left(\frac{x_2}{8} + \frac{3x_6}{80}\right)$	1	41
Werepyre	Magic	1	$\left(\frac{3(x_1 + 35)}{40} + \frac{3(x_3 + 35)}{40} + \frac{5x_6}{80}\right)$	$(0.75)(1.33)$	58
Werepyre	Spell	1.5	$\left(\frac{3(x_1 + 35)}{40} + \frac{3(x_3 + 35)}{40} + \frac{5x_6}{80}\right)$	2	175

Table 5: Total Damage Output with Ability Multipliers from subrace abilities, Stat bonuses from subrace abilities, ToA bonuses, and Ability Multipliers are derived from the official forum [3]. The Base damage formula is derived from the objective function. The damage output is calculated by substituting the decision variables $x_1...x_6$ with the optimal stat distribution (x^*) found previously for each subrace and attack type into objective function.

The results highlight clear distinctions in subrace specialization. Vampires emerge as the strongest in Spell-based combat, achieving the highest overall damage with its optimized spell setup. This conclusion is supported by the powerful +50 INT and CHA buffs and a generous +37.5% spell multiplier which makes Vampire an ideal choice for INT-focused spellcasters, especially those investing heavily in LUK. The Werewolf subrace, in contrast, dominates in physical combat, with the highest melee and ranged damage values. This reflects its raw +50 STR and DEX buffs and the efficiency of focusing purely on offensive stats. Werepyre, intended as a hybrid class, exhibits the highest performance in magic-based attacks, but its split reliance on both STR and INT dilutes its melee attack damage. Despite being weaker than Werewolf or Vampire in Melee and Ranged attack types, Werepyre maintains reasonably strong performance across both Melee and Magic. This duality may offer strategic advantages in actual gameplay scenarios where, despite us assuming that enemies have 0% resistance or blocking, enemies have high resistance or blocking rates against a specific damage type. For instance, Werepyres can pivot between physical and magical weapons based on the encounter, giving them a situational edge in battles that penalize pure physical or magical builds.

In fact, if we compare the post-multiplier damage results with the pre-multiplier damage, intentional design choices become clear in how each Darkovian subrace is meant to specialize. For example, as shown in Table 3, Vampire has relatively balanced base damage across Melee, Ranged, and Magic with base values around 72 and a slightly higher 84 for Ranged. However, when ability multipliers are applied, only Spell damage scales dramatically, reaching a significantly higher post-multiplier value.

In effect, the way post-multiplier damage diverges from base damage confirms that each subrace is designed to excel in a distinct combat style: Vampire in **spellcasting**, Werewolf in **physical aggression**, and Werepyre in **adaptability**.

To gain deeper insight into the value of each stat and guide more informed decision-making, we formulate the dual of our optimization problem. While the primal problem determines the best stat allocation to maximize damage under a fixed budget, the dual reveals the marginal value or sensitivity of each constraint- specifically, how much additional damage could be gained per unit increase in available stat points. This is particularly useful for early-game scenarios where character levels are not as high so stat points are limited and players must prioritize certain stats over others. By analyzing the dual variables, we can identify which stats provide the greatest return on investment, helping players make optimal choices even when a full allocation is not yet possible.

To formulate the dual of our problem, we first relax the integer constraint and treat our original LP problem as the primal. The primal is in inequality form, minimizing a linear objective

function subject to linear inequality constraints. Each inequality constraint in the primal corresponds to a dual variable, and the dual problem maximizes a linear combination of the right-hand sides of these constraints, weighted by those dual variables. The dual constraints arise from transposing the primal's constraint matrix and setting it equal to the negation of the primal objective coefficients, since our objective is to minimize. In matrix/vector form, the dual problem is:

$$\begin{aligned} \min_{\lambda \in \mathbb{R}^{13}} \quad & - \begin{bmatrix} 750 \\ \mathbf{0}_{6 \times 1} \\ 250 \cdot \mathbf{1}_{6 \times 1} \end{bmatrix}^T \lambda \\ \text{subject to} \quad & [\mathbf{1}_{6 \times 1} \quad -I_{6 \times 6} \quad I_{6 \times 6}] \lambda \geq -\mathbf{c} \\ & \lambda \geq 0 \end{aligned}$$

The c vectors remain the same as those in the table shown previously. Solving the dual provides valuable information as each dual variable (shadow price) reflects the rate of improvement in the objective per unit increase in its associated constraint bound. Note that λ is a 13-dimensional vector, where: λ_1 represents the total strat constraint, $\lambda_2 \dots \lambda_7$ represent the lower bound on each of the stats (STR, DEX, INT, END, CHA, LUK), and $\lambda_8 \dots \lambda_{13}$ represents the upper bound. The dual variables and objective function value for each subrace and attack type are displayed in the table below:

Subrace	Type	λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7	λ_8	λ_9	λ_{10}	λ_{11}	λ_{12}	λ_{13}	Damage (ν^*)
Vampire	Melee	0	0	0	0	0	0	0	0.25	0	0	0	0	0.04	72
Vampire	Ranged	0.04	0	0	0	0	0	0	0.16	0.06	0	0	0	0.04	84
Vampire	Magic	0	0	0	0	0	0	0	0	0	0.250	0	0	0.04	72
Vampire	Spell	0	0	0	0	0	0	0	0	0	0.250	0	0	0.04	72
Werewolf	Melee	0	0	0	0	0	0	0	0.250	0	0	0	0	0.04	72
Werewolf	Ranged	0.04	0	0	0	0	0	0	0.16	0.06	0	0	0	0.04	84
Werewolf	Magic	0	0	0	0	0	0	0	0	0	0.025	0	0	0.04	72
Werewolf	Spell	0	0	0	0	0	0	0	0	0	0.25	0	0	0.04	72
Werepyre	Melee	0.04	0	0	0	0	0	0	0.03	0	0.03	0	0	0	41
Werepyre	Ranged	0	0	0	0	0	0	0	0	0.13	0	0	0	0.4	41
Werepyre	Magic	0.06	0	0	0	0	0	0	0.01	0	0.01	0	0	0	53
Werepyre	Spell	0.06	0	0	0	0	0	0	0.01	0	0.01	0	0	0	53

Table 6: Dual variable values for each subrace and weapon type along with the objective function value. Each dual variable represents the marginal cost in the optimal objective function value for an increase in the corresponding primal variable.

Note that the objective function value of the dual is the same as the objective function value of the primal problem- indicating that strong duality holds and confirming that the dual and primal

are properly formulated. The analysis of the dual problem allows us to interpret the strategic value of each constraint, e.g. a nonzero dual value highlights where additional stat points would most improve the performance of the player. For example, in the row corresponding to Werepyre using a Magic Attack, we observe that $\lambda_1 = 0.06$, $\lambda_8 = 0.01$ and $\lambda_{10} = 0.01$. Since λ_1 corresponds to the total stat cap and λ_8 and λ_{10} correspond to STR and INT, this indicates that the optimal solution is limited by these three constraint, i.e, STR and INT are both maxed out at their 250-point cap, and the total stat budget of 750 is fully used which matches the LP solution. Increasing any of these bounds would lead to an increase in the objective or specifically approximately 0.06 more damage per extra point of total stat, and 0.01 more damage per extra point of STR or INT. Thus the stats that are most valuable to invest for a Werepyre using magic attacks would be STR and INT as both yield measurable gains when relaxed, suggesting they are key contributors to maximizing magic-based damage for this subrace.

On the other hand, looking at Vampire Ranged, we see that $\lambda_8 = 0.16$ and $\lambda_9 = 0.06$, which means that if the player had extra stat points and was able to allocate them, to maximize return on investment, it would be the most beneficial in terms of damage output to allocate these stats into STR as opposed to DEX.

These results reveal insight to building characters especially at lower levels. For Werepyres at lower levels using Magic attacks, it should be a priority to invest in STR and INT first, since they directly constrain optimal damage. For Vampire's using Ranged attacks, we mentioned earlier that investing in STR is first priority. Additionally, because each level increases the total stat cap by 5 points, the positive value of λ_1 in both examples implies that having more capacity to allocate stat points can yield immediate improvements in combat effectiveness as long as those additional points are invested optimally. This highlights that not only is stat allocation important, but also that simply increasing your available pool by leveling expands your potential to improve damage, especially when directed toward high-impact stats like STR and INT in this case. Gameplay-wise, this is great motivation for players to grind out XP to level up and increase their total stat pool.

Extension

To more realistically model combat scenarios in AdventureQuest, we extended our original problem formulation to account for multi-turn battles and the use of buffs from pets, guests, and player abilities. For simplicity, this will be modeled as a 2-turn battle, during which the first turn is dedicated to buffing their damage followed by a turn of damage output. This reflects a common strategic pattern in gameplay, where front-loaded buffs are applied early to maximize their benefit across subsequent turns. As a result, the objective function is modified to maximize the total damage over two turns of combat, incorporating both base damage and a multiplier that models the effect of buffs and supports scaling.

In this extended formulation, we incorporate two of the most commonly used support units in AdventureQuest: Angra Linnorm and Poelala. While Poelala is available in both pet and guest forms, Angra Linnorm is only available as a pet. To reflect a typical and practical combat setup, we assume Poelala is used in the guest slot and Angra Linnorm occupies the pet slot.



Figure 9 : Angra Linnorm on the left and Poelala on the right

These support units can be obtained in-game with moderate effort. Angra Linnorm is available as a pet through the Haunted Dragonlord Set quest, while Poelala is obtained via AdventureQuest's Golden Gift Box system. Specifically, the Poelala pet is classified as an Ultra Rare reward with approximately a 1-in-33 chance of being unboxed [12]. Gift Boxes require Z-Tokens, a premium currency that can be earned in-game or purchased with real money. The Poelala guest, in contrast, is directly purchasable for 3124 Z-Tokens, making it more accessible to players who believe gambling is a sin.

Although it is technically possible to run double Poelala (as both pet and guest) for additional damage amplification, we do not model this scenario in our analysis. This simplification ensures a balanced treatment across all builds while capturing the typical tradeoff between buffing options for melee/ranged and magic strategies. More importantly, most players do not have access to both Poelala as both guest and pet since this requires significant investment with Z-tokens which require either a long time to gather in-game or an *expensive* credit card payment. The selection is thus fixed to one pet and one guest, chosen to represent a realistic and strategically viable two-turn combat plan.



Figure 10: A player utilizing double Poelala. The Poelala on the right is the guest and Poelala on the left is the pet. The pet Poelala is currently in the casting animation to boost magic and spell damage

Maintaining the same assumptions used in the original problem formulation, we now include some additional assumptions to model this extended version:

- 1) Two-turn battle structure where the first turn is used solely to apply buffs (no damage dealt) and during the second turn, the player attacks with all buffs fully active
- 2) The player will use Poelala as the guest and Angra Linnorm as the pet. Both support units are assumed to be at level 150, matching the player's level. Lower level versions of the units are also acquirable albeit with smaller multipliers for balancing purposes.

- 3) We assume all buffs are independent, meaning buffs from pets and guests are assumed to stack multiplicatively without any interaction effects.

Thus our problem formulation in words is now:

Maximize: The basic attack damage from one Darkovian subrace in two turns with the first turn solely dedicated to buffing and the second turn to attacking

Subject to:

- 1) Total stat point allocation cannot be below 0 or exceed 750.
- 2) Point allocation towards each stat cannot be below 0 or exceed 250.
- 3) Stat points are integers and can only be incremented by 5

To incorporate the effects of Poelala and Angra Linnorm into our optimization model, we translate their in-game stat-based buff mechanics into explicit multiplicative damage modifiers in the objective function. These buffs depend on the player's stat (such as STR, DEX, or INT) and CHA, and scale linearly based on their relative values. We model each buff as a continuous function that outputs a multiplier applied to the base stat-driven damage. These multipliers are incorporated into the objective function to reflect the second-turn damage boost that results from activating these pets during the first turn. The following expressions describe how we mathematically represent the contribution of each buff:

Poelela:

Instead of attacking normally, this pet boosts all Magic attacks made by the player by:
 $+ (12 + 12 \cdot (0.75 \cdot (YourINT/ExpINT) + 0.25 \cdot (YourCHA/ExpCHA)) \cdot 4/3)\%$
 This includes all normal player attacks with a Magic weapon, all specials made with a Magic weapon and all attacks in a Magic-locked armour. Magic spells gain 3/8 of the boost.

Note that $ExpINT$ and $ExpCHA$ are the maximum stat points you can have in that particular stat. In this case, 250, since this reflects the assumed stat cap for a fully trained character [13].

In our objective function, this translates to a multiplier of

$$M_p(x) = 1 + \frac{4}{3} (0.12 + 0.12(0.75 \cdot \frac{x_3}{250} + 0.25 \cdot \frac{x_5}{250})) = 1.16 + \frac{0.12x_3}{250} + \frac{0.04x_5}{250}$$

Angra Linnorm:

Instead of dealing damage, Angra Linnorm boosts Melee/Ranged damage, and provides a Backlash effect.

Boosts +12% to Melee and Ranged damage from your weapons and spells, multiplied by:
 $(0.5 + 0.25 \cdot (Weapon\ Main\ Stat)/VStat + 0.25 \cdot (CHA/VStat))$

This value is scaled by STR/DEX and CHA. Without CHA, the boost value is +9%. If using a Magic weapon, no damage boost is applied.

Note that *Weapon Main Stat* refers to the corresponding stat to the attack type, i.e STR for Melee and DEX for Ranged. *VStat* is the same as *ExpINT* and *ExpCHA* from before and so is also 250 [14].

In our objective function, this translates to a multiplier of:

$$M_A(x) = 1 + 0.12(0.5 + 0.25 \cdot \frac{x_i}{250} + 0.25 \cdot \frac{x_5}{250}) = 1.06 + \frac{0.03x_i}{250} + \frac{0.03x_5}{250}$$

where x_i is either x_1 or x_3 depending on type of attack.

Although Angra Linnorm and Poelala do not stack or combine their effects across attack types, they represent two of the most commonly used support units in optimal loadouts. Because players typically focus on one type of attack in any given battle, we model damage for each attack type separately, assuming the appropriate buff source is used.

Thus our new general problem formulation looks like this noting that we multiply by (-1) to convert our maximization problem into an equivalent minimization:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{Z}^6} \quad & f(\mathbf{x}) = -M(\mathbf{x}) \cdot \left(\sum_{i=1}^6 c_i x_i \right) \\ \text{subject to} \quad & \sum_{i=1}^6 x_i \leq 750 \\ & 0 \leq x_i \leq 250 \quad \text{for } i = 1, \dots, 6 \\ & x_i \in \{0, 5, 10, \dots, 250\} \quad \text{for } i = 1, \dots, 6 \end{aligned}$$

where $M(x)$ is $M_P(x)$ or $M_A(x)$ depending on attack type. However we see when constructing our problem formulation that including these multipliers transforms the problem into an Integer Non-linear Programming (INLP) problem since our pet/guest multiplier contains decision variables. For each subproblem, the constraints remain the same but the resulting objective function changes depending on $M(x)$. Below is a table that displays all the objective functions:

Subrace	Attack Type	Objective Function $f(\mathbf{x})$
Vampire	Melee	$\left(1.06 + \frac{0.03}{250}x_1 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_1}{4} + \frac{3x_6}{80}\right)$
	Ranged	$\left(1.06 + \frac{0.03}{250}x_2 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_1}{5} + \frac{x_2}{10} + \frac{3x_6}{80}\right)$
	Magic	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$
	Spell	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$
Werewolf	Melee	$\left(1.06 + \frac{0.03}{250}x_1 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_1}{4} + \frac{3x_6}{80}\right)$
	Ranged	$\left(1.06 + \frac{0.03}{250}x_2 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_1}{5} + \frac{x_2}{20} + \frac{3x_6}{80}\right)$
	Magic	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$
	Spell	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$
Werepyre	Melee	$\left(1.06 + \frac{0.03}{250}x_1 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_1 + x_3}{16} + \frac{3x_6}{80}\right)$
	Ranged	$\left(1.06 + \frac{0.03}{250}x_2 + \frac{0.03}{250}x_5\right) \cdot \left(\frac{x_2}{8} + \frac{3x_6}{80}\right)$
	Magic	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{3x_1 + 3x_3}{40} + \frac{5x_6}{80}\right)$
	Spell	$\left(1.16 + \frac{0.12}{250}x_3 + \frac{0.04}{250}x_5\right) \cdot \left(\frac{x_3}{4} + \frac{3x_6}{80}\right)$

Table 7: New objective functions for each subrace and attack type with core-stat-damage along with Pet/Guest provided Multipliers

Each objective function appears non-linear due to the multiplicative interaction between core-stat damage and pet/guest multipliers, however it can be equivalently rewritten in quadratic form:

$$\begin{aligned}
& \min_{\mathbf{x} \in \mathbb{Z}^6} f(\mathbf{x}) = -(\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}) \\
& \text{subject to } \sum_{i=1}^6 x_i \leq 750 \\
& \quad 0 \leq x_i \leq 250, \quad \text{for } i = 1, \dots, 6 \\
& \quad x_i \in \{0, 5, 10, \dots, 250\}
\end{aligned}$$

We can create a table to contain all the Q matrices and c vectors as shown below:

Subrace	Attack Type	Q Matrix						c Vector	
Vampire	Melee		$3.0 \cdot 10^{-5}$	0	0	0	$3.0 \cdot 10^{-5}$	$4.5 \cdot 10^{-6}$	0.265
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			$3.0 \cdot 10^{-5}$	0	0	0	0	$4.5 \cdot 10^{-6}$	0
		$4.5 \cdot 10^{-6}$	0	0	0	$4.5 \cdot 10^{-6}$	0	0.03975	
Vampire	Ranged		0	$2.4 \cdot 10^{-5}$	0	0	$2.4 \cdot 10^{-5}$	0	0.212
			$2.4 \cdot 10^{-5}$	$1.2 \cdot 10^{-5}$	0	0	$1.2 \cdot 10^{-5}$	$4.5 \cdot 10^{-6}$	0.106
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			$2.4 \cdot 10^{-5}$	$1.2 \cdot 10^{-5}$	0	0	0	$4.5 \cdot 10^{-6}$	0
		0	$4.5 \cdot 10^{-6}$	0	0	$4.5 \cdot 10^{-6}$	0	0.03975	
Vampire	Magic		0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0.00012	0	$4.0 \cdot 10^{-5}$	$1.8 \cdot 10^{-5}$	0.29
			0	0	0	0	0	0	0
			0	0	$4.0 \cdot 10^{-5}$	0	0	$6.0 \cdot 10^{-6}$	0
		0	$1.8 \cdot 10^{-5}$	0	$6.0 \cdot 10^{-6}$	0	0.0435		
Vampire	Spell		0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0.00012	0	$4.0 \cdot 10^{-5}$	$1.8 \cdot 10^{-5}$	0.29
			0	0	0	0	0	0	0
			0	0	$4.0 \cdot 10^{-5}$	0	0	$6.0 \cdot 10^{-6}$	0
		0	$1.8 \cdot 10^{-5}$	0	$6.0 \cdot 10^{-6}$	0	0.0435		
Werewolf	Melee		$3.0 \cdot 10^{-5}$	0	0	0	$3.0 \cdot 10^{-5}$	$4.5 \cdot 10^{-6}$	0.265
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			$3.0 \cdot 10^{-5}$	0	0	0	0	$4.5 \cdot 10^{-6}$	0
		$4.5 \cdot 10^{-6}$	0	0	0	$4.5 \cdot 10^{-6}$	0	0.03975	
Werewolf	Ranged		0	$2.4 \cdot 10^{-5}$	0	0	$2.4 \cdot 10^{-5}$	0	0.212
			$2.4 \cdot 10^{-5}$	$6.0 \cdot 10^{-6}$	0	0	$6.0 \cdot 10^{-6}$	$4.5 \cdot 10^{-6}$	0.053
			0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			$2.4 \cdot 10^{-5}$	$6.0 \cdot 10^{-6}$	0	0	0	$4.5 \cdot 10^{-6}$	0
		0	$4.5 \cdot 10^{-6}$	0	0	$4.5 \cdot 10^{-6}$	0	0.03975	
Werewolf	Magic		0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0.00012	0	$4.0 \cdot 10^{-5}$	$1.8 \cdot 10^{-5}$	0.29
			0	0	0	0	0	0	0
			0	0	$4.0 \cdot 10^{-5}$	0	0	$6.0 \cdot 10^{-6}$	0
		0	$1.8 \cdot 10^{-5}$	0	$6.0 \cdot 10^{-6}$	0	0.0435		
Werewolf	Spell		0	0	0	0	0	0	0
			0	0	0	0	0	0	0
			0	0	0.00012	0	$4.0 \cdot 10^{-5}$	$1.8 \cdot 10^{-5}$	0.29
			0	0	0	0	0	0	0
			0	0	$4.0 \cdot 10^{-5}$	0	0	$6.0 \cdot 10^{-6}$	0
		0	$1.8 \cdot 10^{-5}$	0	$6.0 \cdot 10^{-6}$	0	0.0435		
Werepyre	Melee		$7.5 \cdot 10^{-6}$	0	$7.5 \cdot 10^{-6}$	0	$7.5 \cdot 10^{-6}$	$4.5 \cdot 10^{-6}$	0.06625
			0	0	0	0	0	0	0
			$7.5 \cdot 10^{-6}$	0	0	0	$7.5 \cdot 10^{-6}$	0	0.06625
			0	0	0	0	0	0	0
			$7.5 \cdot 10^{-6}$	0	$7.5 \cdot 10^{-6}$	0	0	$4.5 \cdot 10^{-6}$	0
		$4.5 \cdot 10^{-6}$	0	0	0	$4.5 \cdot 10^{-6}$	0	0.03975	

Subrace	Attack Type	Q Matrix	c Vector
Werepyre	Ranged	$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1.5 \cdot 10^{-5} & 0 & 0 & 1.5 \cdot 10^{-5} & 4.5 \cdot 10^{-6} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1.5 \cdot 10^{-5} & 0 & 0 & 0 & 4.5 \cdot 10^{-6} \\ 0 & 4.5 \cdot 10^{-6} & 0 & 0 & 4.5 \cdot 10^{-6} & 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0.1325 \\ 0 \\ 0 \\ 0 \\ 0.03975 \end{bmatrix}$
Werepyre	Magic	$\begin{bmatrix} 0 & 0 & 3.6 \cdot 10^{-5} & 0 & 1.2 \cdot 10^{-5} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 3.6 \cdot 10^{-5} & 0 & 3.6 \cdot 10^{-5} & 0 & 1.2 \cdot 10^{-5} & 3.0 \cdot 10^{-5} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 1.2 \cdot 10^{-5} & 0 & 1.2 \cdot 10^{-5} & 0 & 0 & 1.0 \cdot 10^{-5} \\ 0 & 0 & 3.0 \cdot 10^{-5} & 0 & 1.0 \cdot 10^{-5} & 0 \end{bmatrix}$	$\begin{bmatrix} 0.087 \\ 0 \\ 0.087 \\ 0 \\ 0 \\ 0.0725 \end{bmatrix}$
Werepyre	Spell	$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.00012 & 0 & 4.0 \cdot 10^{-5} & 1.8 \cdot 10^{-5} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4.0 \cdot 10^{-5} & 0 & 0 & 6.0 \cdot 10^{-6} \\ 0 & 0 & 1.8 \cdot 10^{-5} & 0 & 6.0 \cdot 10^{-6} & 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0.29 \\ 0 \\ 0 \\ 0.0435 \end{bmatrix}$

Table 8: Resulting Q matrices and c vectors for each subrace and attack type for each respective QP subproblem.

As mentioned previously, this problem is now an INLP or more specifically an Quadratic Integer Programming (QIP). Similar to our initial problem, the feasible region Ω is the set of all 6-dimensional integer vectors $\mathbf{x}$ representing valid stat allocations, where each stat can take values from 0 to 250 in increments of 5, and the total stat budget does not exceed 750 points. Analogously, the feasible set is finite and non-empty. This means that a minimum value of the quadratic objective function must exist and be attained. Because the feasible region is not continuous, both FONC and SONC are not applicable in this problem but we similarly consider a relaxation of the QP by dropping the integer multiples of 5 constraints on the decision variables:

$$\begin{aligned}
& \min_{\mathbf{x} \in \mathbb{Z}^6} f(\mathbf{x}) = -(\mathbf{x}^T Q \mathbf{x} + \mathbf{c}^T \mathbf{x}) \\
& \text{subject to} \quad \sum_{i=1}^6 x_i \leq 750 \\
& \quad \quad \quad 0 \leq x_i \leq 250, \quad \text{for } i = 1, \dots, 6
\end{aligned}$$

Here the relaxed feasible set is again a compact polytope in $\mathbb{R}^6$ and contained within a Euclidean ball of radius 800 centered at the origin. Similar to before, since our objective function is continuous, we can guarantee on our feasible set that the objective function attains both a maximum and minimum from Weierstrass' Theorem. Additionally, since the feasible region is a polytope defined by linear constraints, the minimum will occur on the boundary of the feasible set. That is, there exists an $\mathbf{x}^* \in \Omega$ such that:

$$f(x^*) \leq f(x) \quad \text{for all } x \in \Omega$$

Since each matrix Q is symmetric, the Hessian of the objective function is simply $2Q$. To check if each Hessian matrix is Positive Definite (PD), we apply Sylvester's criterion. A quick inspection reveals that each matrix Q is not PD, and thus our problem is not strictly convex, but still convex. As a result, we cannot guarantee that the global minimizer is unique i.e, multiple global optima may exist.

However, each Hessian matrix is Positive Semi-Definite (PSD) which means the SONC is satisfied. This confirms that any point satisfying the FONC for the relaxed QP is in fact a global minimizer.

QPs extend linear programs by introducing a quadratic objective function while retaining linear constraints. For convex QPs, the most common solution method is to use interior point methods, a generalization of its linear programming counterpart. This approach iteratively moves through the interior of the feasible region toward the optimum by solving a sequence of linear systems derived from the KKT conditions, efficiently handling both the quadratic curvature of the objective and the polyhedral constraint structure [15]. When integer constraints are introduced, as in QIP, the problem becomes non-convex and combinatorial and so solvers typically apply a branch-and-bound framework operating in a similar manner to solving LP's as mentioned before

Below is a table of the results and the optimal stat distributions for each subrace and attack type.

Subrace	Attack	STR	DEX	INT	END	CHA	LUK	Damage (ν^*)	N/A
Vampire	Melee	250	0	0	0	250	250	83	DEX, INT, END
Vampire	Ranged	250	250	0	0	0	250	94	INT, END, CHA
Vampire	Magic	0	0	250	0	0	250	99	STR, DEX, END, CHA
Vampire	Spell	0	0	250	0	0	250	99	STR, DEX, END, CHA
Werewolf	Melee	250	0	0	0	250	250	83	DEX, INT, END
Werewolf	Ranged	250	250	0	0	0	250	80	INT, END, CHA
Werewolf	Magic	0	0	250	0	250	250	99	STR, DEX, END
Werewolf	Spell	0	0	250	0	250	250	99	STR, DEX, END
Werepyre	Melee	250	0	0	0	0	250	49	DEX, INT, END, CHA
Werepyre	Ranged	0	250	0	0	0	250	47	STR, INT, END, CHA
Werepyre	Magic	250	0	250	0	0	250	70	DEX, END, CHA
Werepyre	Spell	250	0	250	0	0	250	99	DEX, END, CHA

Table 9: Optimal Stat Distributions after Pet/Guest Multipliers before multipliers for QIP

In a similar and remarkable manner to before, the solutions for the QIP and QP formulations are identical in every case, both in terms of the optimal objective value and the stat allocation

vectors. This indicates that the relaxed solutions happened to lie at integer multiples of 5 regardless of the constraint. This alignment arises because of the structure of the objective function which are sparse, diagonal-heavy matrices. These naturally encourage the optima to fall on boundaries of the feasible polytope, particularly at extreme values like 0 or 250 [16].

Subrace	Attack	STR	DEX	INT	END	CHA	LUK	Damage (ν^*)	N/A
Vampire	Melee	250	0	0	0	250	250	83	DEX, INT, END
Vampire	Ranged	250	250	0	0	0	250	94	INT, END, CHA
Vampire	Magic	0	0	250	0	0	250	99	STR, DEX, END, CHA
Vampire	Spell	0	0	250	0	0	250	99	STR, DEX, END, CHA
Werewolf	Melee	250	0	0	0	250	250	83	DEX, INT, END
Werewolf	Ranged	250	250	0	0	0	250	80	INT, END, CHA
Werewolf	Magic	0	0	250	0	250	250	99	STR, DEX, END
Werewolf	Spell	0	0	250	0	250	250	99	STR, DEX, END
Werepyre	Melee	250	0	0	0	0	250	49	DEX, INT, END, CHA
Werepyre	Ranged	0	250	0	0	0	250	47	STR, INT, END, CHA
Werepyre	Magic	250	0	250	0	0	250	70	DEX, END, CHA
Werepyre	Spell	250	0	250	0	0	250	99	DEX, END, CHA

Table 10: Optimal Stat Distributions after Pet/Guest Multipliers before multipliers for QP

After incorporating pets and guests into the model, we observe that all subraces now fully utilize the entire 750-point stat budget, whereas previously only the Werepyre builds had done so. Furthermore, the base damage ν^* has increased significantly across every subrace and attack type. This highlights the role of pets and guests in gameplay, providing a significant boost to damage output even without requiring additional stat investment as in the case of Werepyres.

The QP formulation follows the same stat allocation framework as the LP, where each decision variable represents how many points to assign to STR, DEX, INT, END, CHA, and LUK, in increments of 5, with a total cap of 750 points. However, instead of maximizing a linear damage function, the objective now involves a convex quadratic function.. For each case, a separate Pyomo model was built using a quadratic objective matrix and linear constraints, and solved using Gurobi for its QP solving capabilities. Gurobi primarily uses primal-dual interior-point methods to solve convex QPs and branching for integer constraints [16]. Due to the small, discrete feasible space and the convex structure of the objective, Gurobi returned optimal solutions in under 0.01 seconds on average.

Now we then apply the same ability multipliers used in the LP formulation to convert the optimized stat distribution into total expected damage for each subrace and attack type.

Subrace	Attack Type	Damage
Vampire	Melee	62
Vampire	Ranged	60
Vampire	Magic	56
Vampire	Spell	285
Werewolf	Melee	73
Werewolf	Ranged	71
Werewolf	Magic	47
Werewolf	Spell	261
Werepyre	Melee	49
Werepyre	Ranged	46
Werepyre	Magic	76
Werepyre	Spell	159

Table 11: Total damage output with Pets/Guests after ability multipliers for each subrace and attack type. The same ability and ToA multipliers from Table 4 were used. Base damage formulas were modified to include the multipliers as shown in Table 6.

After applying the ability and type multipliers, we observe significant amplification in total damage across all subrace and attack type combinations compared to our scenario without pets or guests. In particular, Vampire Spellcasters have the highest damage outputs of 285, further emphasizing their design as specialized spellcasters. Notably, Werepyre, which was previously underwhelming in the LP formulation, now shows competitive performance in magic and spell-based builds, thanks to its flexible stat synergy and enhanced core multipliers. In fact, Werepyre magic attacks have a damage output of 76- the highest of all the non-spell attacks across all subraces whereas previously in our LP problem, it was Werewolf melee and range. On the other hand, we see that Vampire Melee and Ranged remain relatively low in output even after multipliers, reinforcing that their design favors INT-based builds.

We also see the importance of ability multipliers, as they can significantly shift the perceived effectiveness of subrace-attack combinations. Prior to applying these bonuses, Werepyres consistently exhibited the lowest damage output across all attack types as shown in Table 7. However, after incorporating the appropriate multipliers, their performance becomes much more competitive, particularly in Magic attack builds, while also offering added versatility to Melee attacks due to their hybrid scaling. This highlights that, despite weaker baseline damage, Werepyres can be a strong choice when fully accounting for gameplay mechanics.

Conclusion

In this project, we explored the optimal allocation of stat points for each of the Darkovian subraces and attack types in Adventure Quest, with the objective of maximizing our total damage output. By formulating the problem as both a simplified LP problem and a more refined QP problem, we were able to compare the damage scaling under simplified linear assumptions and more realistic nonlinear interactions. The simple LP problem solely utilized stat damage formulas and Darkovian subrace abilities to calculate the damage output, disregarding any other gameplay mechanics. To more accurately model the gameplay experience, our extended model incorporated pets and guests which transformed the simple LP into a QP. The resulting optimal stat allocations revealed that all subraces fully exhausted the 750-point cap, in contrast to the LP model where only Werepyres did so. This indicates that when pets and guests are taken into account, investing in every available stat point becomes more valuable, especially in secondary stats such as CHA. Importantly, we found that applying ability and type multipliers significantly altered the ranking and effectiveness of certain builds. Subraces like the Werepyre, which originally showed poor performance under both LP and base QP models, rose to competitive levels once all gameplay bonuses were considered. Meanwhile, Vampires continued to excel in spell-based attacks, reaffirming their role as dedicated spellcasters. These findings demonstrate the nuanced balance of the game and the importance of accounting for layered mechanics when making optimization-based decisions.

While our current model captures the fundamental tradeoffs between stats and damage output, it remains a simplified abstraction of the full game environment. One limitation is that it does not currently include style bonuses, a mechanic that rewards players for investing at least 150 points into a particular stat by providing further damage bonuses. One of this project's assumptions was that the player's character would be at max level thus lower character levels, which would reduce the available stat cap, were not modeled.

In future work, it would be valuable to expand the model to capture these additional game features as well as items, and in-battle mechanics such as elemental affinities or enemy resistances are not yet represented in the model. For example, modeling style bonuses could involve conditional multipliers based on the amount invested, introducing a layer of combinatorial logic to the problem formulation. Lower-level character scenarios could be simulated by capping the stat total below 750 or specific stats under a certain value, yielding insights into mid-game progression. Finally, incorporating turn-based resource consumption, e.g. SP and MP usage for abilities or pets, would move the model closer to a full-fledged combat simulator and make the optimization framework even more practical for gameplay planning.

References

All listed references were accessed most recently on May 11, 2025

[1] Adventure Quest Review - Web archive of game review forum of Adventure Quest in 2005.
https://web.archive.org/web/20110715001909/http://archive.omgn.com/reviews.php?Item_ID=35&Offset=10

[2] Big List of AQ Formulas - Comprehensive list of core AdventureQuest formulas used to understand and model game behavior.
https://adventurequestwiki.fandom.com/wiki/Big_List_of_AQ_Formulas

[3] Subraces and Classes - Comprehensive list of Darkovian Subraces and each subrace's abilities.
https://adventurequestwiki.fandom.com/wiki/Subraces_and_Classes

[4] Vampire Subrace - Official forum post detailing the abilities, mechanics, and progression of the Vampire subrace in AdventureQuest.
<https://forums2.battleon.com/f/tm.asp?m=553400>

[5] Werewolf Subrace - Official forum post detailing the abilities, mechanics, and progression of the Werewolf subrace in AdventureQuest.
<https://forums2.battleon.com/f/tm.asp?m=18958>

[6] Werepyre Subrace and Armors - Forum post covering the mechanics and abilities of the Werepyre subrace as well as its associated armors in AdventureQuest.
<https://forums2.battleon.com/f/tm.asp?m=22293206&mpage=4&key=�>

[7] Werepyre Subrace Hybrid Nature- Forum post clarifying the hybrid nature of the Werepyre subrace as well as the hybrid damage formula it uses.
<https://forums2.battleon.com/f/printable.asp?m=22393411>

[8] Simplex Method - Short pdf containing technical notes on the Simplex Method by M. Casquilho, University of Lisbon.
https://web.tecnico.ulisboa.pt/mcasquilho/compute/_linpro/TaylorB_module_c.pdf

[9] Linear Programming - Introduction and applications overview post for Linear Programming by GeeksforGeeks.
<https://www.geeksforgeeks.org/linear-programming/>

[10] Advanced Branch-and-Bound Techniques in MILP - Overview of MILP solving techniques, by T. Ralphs Lehigh University.

<https://coral.ise.lehigh.edu/~ted/files/computational-mip/lectures/Lecture11.pdf>

[11] GNU Linear Programming Kit (GLPK) - Official documentation of the GLPK package used for solving large-scale linear programming.

<https://www.gnu.org/software/glpk/#TOCdocumentation>

[12] Golden Giftboxes - Overview of the Golden Giftbox feature in AdventureQuest, a loot box-style mechanic similar to those found in other video games.

https://adventurequestwiki.fandom.com/wiki/Golden_Giftboxes#Weapons

[13] Poelala - Official forum post detailing the mechanics and specific boosting formulas of the Poelala pet/guest.

<https://forums2.battleon.com/f/tm.asp?m=459523>

[14] Angra Linnorm and other Pets - Official forum post detailing the mechanics and specific boosting formulas of the Angra Linnorm pet.

<https://forums2.battleon.com/f/tm.asp?m=22317138&mpage=5&key=�>

[15] Interior Point Methods for NLP - Overview of interior-point methods and their application to nonlinear programming by Cornell University Optimization Wiki.

https://optimization.cbe.cornell.edu/index.php?title=Interior-point_method_for_NLP

[16] Quadratic Optimization - Overview of Quadratic programming, solution methods, and features, published by Gurobi Optimization.

https://cdn.gurobi.com/wp-content/uploads/quadratic_optimization.pdf

Appendix

```

1  from pyomo.environ import *
2  import numpy as np
3  import time
4
5  #---- Single Turn Damage Optimization Code----
6
7  #Define the stat weight vectors: [STR, DEX, INT, END, CHA, LUK]
8  damage_map = {
9      'vampire': {
10         1: [1/4, 0, 0, 0, 0, 3/80], #melee
11         2: [1/5, 1/10, 0, 0, 0, 3/80], #ranged
12         3: [0, 0, 1/4, 0, 0, 3/80], #magic
13         4: [0, 0, 1/4, 0, 0, 3/80], #spell
14     },
15     'werewolf': {
16         1: [1/4, 0, 0, 0, 0, 3/80],
17         2: [1/5, 1/20, 0, 0, 0, 3/80],
18         3: [0, 0, 1/4, 0, 0, 3/80],
19         4: [0, 0, 1/4, 0, 0, 3/80],
20     },
21     'werepyre': {
22         1: [1/16, 0, 1/16, 0, 0, 3/80], #hybrid melee
23         2: [0, 1/8, 0, 0, 0, 3/80],
24         3: [3/40, 0, 3/40, 0, 0, 5/80], #hybrid magic
25         4: [3/40, 0, 3/40, 0, 0, 5/80],
26     }
27 }
28 #---- ILP Problem ----
29 stat_labels = ['STR', 'DEX', 'INT', 'END', 'CHA', 'LUK']
30 print("ILP Solutions")
31 print(f"{'Subrace':<10} {'Type':<8} " + " ".join(f"{'s':<5}" for s in stat_labels) + " Damage")
32
33 attack_names = {1: "Melee", 2: "Ranged", 3: "Magic", 4: "Spell"}
34 optimal_stats = {}
35
36 for race in damage_map:
37     for atype in range(1, 5):
38         w = damage_map[race][atype]
39
40         model = ConcreteModel()
41         model.I = RangeSet(1, 6)
42         model.y = Var(model.I, domain=NonNegativeIntegers, bounds=(0, 50))
43         model.x = Expression(model.I, rule=lambda m, i: 5 * m.y[i])
44         model.obj = Objective(expr=sum(w[i-1] * model.x[i] for i in model.I), sense=maximize)
45         model.total = Constraint(expr=sum(model.x[i] for i in model.I) <= 750)
46
47         start = time.time()
48         SolverFactory('glpk').solve(model, tee=False)
49         end = time.time()
50         elapsed = end - start
51
52         x_vals = [int(model.x[i]()) for i in model.I]
53         damage = model.obj()
54
55         print(f"{'race':<10} {'atype':<6} " + " ".join(f"{'x':<6}" for x in x_vals) + f"{'damage':.4f} Time:
56             ↳ {'elapsed':.3f}s")
57
58     #Storing the optimal stat distributions

```

```

58     optimal_stats[(race.capitalize(), attack_names[atype])] = [model.x[i]() for i in model.I]
59
60     #---- LP Relaxation ----
61     print()
62     print("LP Relaxation Solutions")
63     print(f"{'Subrace':<10} {'Type':<6} " + " ".join(f"{s:<6}" for s in stat_labels) + "Damage")
64
65     for race in damage_map:
66         for atype in range(1, 5):
67             w = damage_map[race][atype]
68
69             relaxed_model = ConcreteModel()
70             relaxed_model.I = RangeSet(1, 6)
71             relaxed_model.x = Var(relaxed_model.I, domain=NonNegativeReals, bounds=(0, 250))
72             relaxed_model.obj = Objective(expr=sum(w[i-1] * relaxed_model.x[i] for i in relaxed_model.I),
73             ↪ sense=maximize)
74             relaxed_model.total = Constraint(expr=sum(relaxed_model.x[i] for i in relaxed_model.I) <= 750)
75
76             start = time.time()
77             SolverFactory('glpk').solve(relaxed_model)
78             end = time.time()
79             elapsed = end - start
80
81             x_vals = [relaxed_model.x[i]() for i in relaxed_model.I]
82             damage = relaxed_model.obj()
83             print(f"{'race':<10} {'atype':<6} " + " ".join(f"{x:<6.2f}" for x in x_vals) + f" {damage:.4f}"
84             ↪ Time: {elapsed:.3f}s")
85
86     #----Applying Ability Multipliers----
87
88     x = [model.x[i]() for i in range(1, 7)] #Stats x1 ... x6
89
90     def evaluate_damage(ability_mult, obj_function, toa_mult):
91         return ability_mult * obj_function(x) * toa_mult
92
93     #Dictionary for each subrace/attack type and the formula: Ability Multipliers, objective function,
94     ↪ Attack type
95     damage_cases = {
96         ("Vampire", "Melee"): (0.765625, lambda x: x[0]/4 + 3*x[5]/80, 1),
97         ("Vampire", "Ranged"): (0.765625, lambda x: x[0]/5 + x[1]/20 + 3*x[5]/80, 1),
98         ("Vampire", "Magic"): (0.65625, lambda x: (x[2] + 50)/4 + 3*x[5]/80, 0.75),
99         ("Vampire", "Spell"): (1.25, lambda x: (x[2] + 50)/4 + 3*x[5]/80, 2),
100
101         ("Werewolf", "Melee"): (0.765625, lambda x: (x[0] + 50)/4 + 3*x[5]/80, 1),
102         ("Werewolf", "Ranged"): (0.765625, lambda x: (x[0] + 50)/5 + (x[1] + 50)/20 + 3*x[5]/80, 1),
103         ("Werewolf", "Magic"): (0.65625, lambda x: x[2]/4 + 3*x[5]/80, 0.75),
104         ("Werewolf", "Spell"): (1.375, lambda x: x[2]/4 + 3*x[5]/80, 2),
105
106         ("Werepyre", "Melee"): (1, lambda x: (x[0] + 35)/16 + (x[2] + 35)/16 + 3*x[5]/80, 1),
107         ("Werepyre", "Ranged"): (1, lambda x: x[1]/8 + 3*x[5]/80, 1),
108         ("Werepyre", "Magic"): (1, lambda x: 3*(x[0] + 35)/40 + 3*(x[2] + 35)/40 + 5*x[5]/80, 0.9975),
109         ("Werepyre", "Spell"): (1.5, lambda x: 3*(x[0] + 35)/40 + 3*(x[2] + 35)/40 + 5*x[5]/80, 2),
110     }
111
112     for (race, atype), (ab_mult, formula, toa_mult) in damage_cases.items():
113         x = optimal_stats[(race, atype)]
114         dmg = evaluate_damage(ab_mult, formula, toa_mult)
115         print(f"{'race':<10} {'atype':<6} Damage: {dmg:.4f}")
116
117     #---- Single Turn Damage Optimization Dual Code----

```

```

115
116 damage_map = {
117     'vampire': {
118         1: [1/4, 0, 0, 0, 0, 3/80],      #melee
119         2: [1/5, 1/10, 0, 0, 0, 3/80],   #ranged
120         3: [0, 0, 1/4, 0, 0, 3/80],     #magic
121         4: [0, 0, 1/4, 0, 0, 3/80],     #spell
122     },
123     'werewolf': {
124         1: [1/4, 0, 0, 0, 0, 3/80],
125         2: [1/5, 1/20, 0, 0, 0, 3/80],
126         3: [0, 0, 1/4, 0, 0, 3/80],
127         4: [0, 0, 1/4, 0, 0, 3/80],
128     },
129     'werepyre': {
130         1: [1/16, 0, 1/16, 0, 0, 3/80],   #hybrid melee
131         2: [0, 1/8, 0, 0, 0, 3/80],
132         3: [3/40, 0, 3/40, 0, 0, 5/80],   #hybrid magic
133         4: [3/40, 0, 3/40, 0, 0, 5/80],
134     }
135 }
136
137 print(f"{'Race':<10} {'Type':<6} {'Objective':<10} {'Time (s)'}")
138 print("-" * 40)
139
140 for race in damage_map:
141     for atype in range(1, 5):
142         c = damage_map[race][atype]
143
144         model = ConcreteModel()
145         model.I = RangeSet(1, 6)
146         model.J = RangeSet(1, 13) # 1 total, 6 lower, 6 upper
147
148         model.lmbda = Var(model.J, domain=NonNegativeReals)
149
150         b = [750] + [0]*6 + [250]*6
151
152         model.obj = Objective(expr=sum(b[j-1] * model.lmbda[j] for j in model.J), sense=minimize)
153
154         def dual_con_rule(model, i):
155             return (
156                 model.lmbda[1]                # total constraint
157                 - model.lmbda[1 + i]          # x_i 0
158                 + model.lmbda[7 + i]          # x_i 250
159                 >= c[i - 1]
160             )
161         model.dual_constraints = Constraint(model.I, rule=dual_con_rule)
162
163         start = time.time()
164         result = SolverFactory('glpk').solve(model)
165         end = time.time()
166
167         print(f"{'race':<10} {'atype':<6} {'value(model.obj):<10.4f} {'end - start:.3f}")
168         for j in model.J:
169             print(f"    lambda[{j}] = {value(model.lmbda[j]):.6f}")
170
171         #---- Two Turn Damage Optimization Code with Pets and Guests----
172
173 qp_problems = {('Vampire', 'Melee'): {'Q': np.array([[3.0e-05, 0.0e+00, 0.0e+00, 0.0e+00, 3.0e-05,
↵ 4.5e-06],

```

```

174 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
175 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
176 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
177 [3.0e-05, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06],
178 [4.5e-06, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.265 , 0. , 0.
  ↳ , 0. , 0. , 0.03975])), ('Vampire', 'Ranged'): {'Q': np.array([[0.0e+00, 2.4e-05,
  ↳ 0.0e+00, 0.0e+00, 2.4e-05, 0.0e+00],
179 [2.4e-05, 1.2e-05, 0.0e+00, 0.0e+00, 1.2e-05, 4.5e-06],
180 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
181 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
182 [2.4e-05, 1.2e-05, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06],
183 [0.0e+00, 4.5e-06, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.212 , 0.106 , 0.
  ↳ , 0. , 0. , 0.03975])), ('Vampire', 'Magic'): {'Q': np.array([[0.0e+00, 0.0e+00,
  ↳ 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
184 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
185 [0.0e+00, 0.0e+00, 1.2e-04, 0.0e+00, 4.0e-05, 1.8e-05],
186 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
187 [0.0e+00, 0.0e+00, 4.0e-05, 0.0e+00, 0.0e+00, 6.0e-06],
188 [0.0e+00, 0.0e+00, 1.8e-05, 0.0e+00, 6.0e-06, 0.0e+00]], 'c': np.array([0. , 0. , 0.29 ,
  ↳ 0. , 0. , 0.0435])), ('Vampire', 'Spell'): {'Q': np.array([[0.0e+00, 0.0e+00, 0.0e+00,
  ↳ 0.0e+00, 0.0e+00, 0.0e+00],
189 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
190 [0.0e+00, 0.0e+00, 1.2e-04, 0.0e+00, 4.0e-05, 1.8e-05],
191 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
192 [0.0e+00, 0.0e+00, 4.0e-05, 0.0e+00, 0.0e+00, 6.0e-06],
193 [0.0e+00, 0.0e+00, 1.8e-05, 0.0e+00, 6.0e-06, 0.0e+00]], 'c': np.array([0. , 0. , 0.29 ,
  ↳ 0. , 0. , 0.0435])), ('Werewolf', 'Melee'): {'Q': np.array([[3.0e-05, 0.0e+00,
  ↳ 0.0e+00, 0.0e+00, 3.0e-05, 4.5e-06],
194 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
195 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
196 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
197 [3.0e-05, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06],
198 [4.5e-06, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.265 , 0. , 0.
  ↳ , 0. , 0. , 0.03975])), ('Werewolf', 'Ranged'): {'Q': np.array([[0.0e+00, 2.4e-05,
  ↳ 0.0e+00, 0.0e+00, 2.4e-05, 0.0e+00],
199 [2.4e-05, 6.0e-06, 0.0e+00, 0.0e+00, 6.0e-06, 4.5e-06],
200 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
201 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
202 [2.4e-05, 6.0e-06, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06],
203 [0.0e+00, 4.5e-06, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.212 , 0.053 , 0.
  ↳ , 0. , 0. , 0.03975])), ('Werewolf', 'Magic'): {'Q': np.array([[0.0e+00, 0.0e+00,
  ↳ 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
204 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
205 [0.0e+00, 0.0e+00, 1.2e-04, 0.0e+00, 4.0e-05, 1.8e-05],
206 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
207 [0.0e+00, 0.0e+00, 4.0e-05, 0.0e+00, 0.0e+00, 6.0e-06],
208 [0.0e+00, 0.0e+00, 1.8e-05, 0.0e+00, 6.0e-06, 0.0e+00]], 'c': np.array([0. , 0. , 0.29 ,
  ↳ 0. , 0. , 0.0435])), ('Werewolf', 'Spell'): {'Q': np.array([[0.0e+00, 0.0e+00,
  ↳ 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
209 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
210 [0.0e+00, 0.0e+00, 1.2e-04, 0.0e+00, 4.0e-05, 1.8e-05],
211 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
212 [0.0e+00, 0.0e+00, 4.0e-05, 0.0e+00, 0.0e+00, 6.0e-06],
213 [0.0e+00, 0.0e+00, 1.8e-05, 0.0e+00, 6.0e-06, 0.0e+00]], 'c': np.array([0. , 0. , 0.29 ,
  ↳ 0. , 0. , 0.0435])), ('Werewolf', 'Melee'): {'Q': np.array([[7.5e-05, 0.0e+00,
  ↳ 7.5e-06, 0.0e+00, 7.5e-06, 4.5e-06],
214 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
215 [7.5e-06, 0.0e+00, 0.0e+00, 0.0e+00, 7.5e-06, 0.0e+00],
216 [0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
217 [7.5e-06, 0.0e+00, 7.5e-06, 0.0e+00, 0.0e+00, 4.5e-06],

```



```

[4.5e-06, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.06625, 0.
↪ 0.06625, 0.
↪ 0.
↪ 0.03975])), ('Werepyre', 'Ranged'): {'Q': np.array([[0.0e+00, 0,
↪ 0.0e+00, 0.0e+00, 0, 0],
[0.0e+00, 1.5e-05, 0.0e+00, 0.0e+00, 1.5e-05, 4.5e-06],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[0.0e+00, 1.5e-05, 0.0e+00, 0.0e+00, 0.0e+00, 4.5e-06],
[0.0e+00, 4.5e-06, 0.0e+00, 0.0e+00, 4.5e-06, 0.0e+00]], 'c': np.array([0.
↪ 0.1325, 0.
↪ 0.
↪ 0.03975])), ('Werepyre', 'Magic'): {'Q': np.array([[0.0e+00, 0.0e+00,
↪ 3.6e-05, 0.0e+00, 1.2e-05, 0.0e+00],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[3.6e-05, 0.0e+00, 3.6e-05, 0.0e+00, 1.2e-05, 1.2e-05],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[1.2e-05, 0.0e+00, 1.2e-05, 0.0e+00, 0.0e+00, 1.2e-05],
[0.0e+00, 0.0e+00, 1.2e-05, 0.0e+00, 1.2e-05, 0.0e+00]], 'c': np.array([0.087, 0.
↪ 0.
↪ 0.0725])), ('Werepyre', 'Spell'): {'Q': np.array([[0.0e+00, 0.0e+00,
↪ 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[0.0e+00, 0.0e+00, 1.2e-04, 0.0e+00, 4.0e-05, 1.8e-05],
[0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00, 0.0e+00],
[0.0e+00, 0.0e+00, 4.0e-05, 0.0e+00, 0.0e+00, 6.0e-06],
[0.0e+00, 0.0e+00, 1.8e-05, 0.0e+00, 6.0e-06, 0.0e+00]], 'c': np.array([0.
↪ 0.
↪ 0.29,
↪ 0.
↪ 0.
↪ 0.0435]))}

```

```

stat_labels = ["STR", "DEX", "INT", "END", "CHA", "LUK"]

```

```

optimal_stats_qp = {}

```

```

#---- QIP Problem ----

```

```

def solve_integer_qip(Q, c):
    model = ConcreteModel()
    model.I = RangeSet(1, 6)
    model.x = Var(model.I, domain=NonNegativeIntegers, bounds=(0, 250))
    model.y = Var(model.I, domain=NonNegativeIntegers, bounds=(0, 50)) #integer multiples of 5
    model.mult_of_5 = ConstraintList()
    for i in model.I:
        model.mult_of_5.add(model.x[i] == 5 * model.y[i])
    model.total = Constraint(expr=sum(model.x[i] for i in model.I) <= 750)

    def obj_rule(m):
        return sum(Q[i-1, j-1] * m.x[i] * m.x[j] for i in m.I for j in m.I) + sum(c[i-1] * m.x[i] for i
        ↪ in m.I)
    model.obj = Objective(rule=obj_rule, sense=maximize)

    start = time.time()
    SolverFactory("gurobi").solve(model)
    elapsed = time.time() - start

    x_vals = [model.x[i]() for i in model.I]
    obj_val = model.obj()
    return x_vals, obj_val, elapsed

```

```

#----QP Problem----

```

```

def solve_relaxed_qp(Q, c):
    model = ConcreteModel()
    model.I = RangeSet(1, 6)
    model.x = Var(model.I, domain=NonNegativeReals, bounds=(0, 250))
    model.total = Constraint(expr=sum(model.x[i] for i in model.I) <= 750)

    def obj_rule(m):

```

```

269         return sum(Q[i-1, j-1] * m.x[i] * m.x[j] for i in m.I for j in m.I) + sum(c[i-1] * m.x[i] for i
    ↪ in m.I)
270 model.obj = Objective(rule=obj_rule, sense=maximize)
271
272 start = time.time()
273 SolverFactory("gurobi").solve(model)
274 elapsed = time.time() - start
275
276 x_vals = [model.x[i]() for i in model.I]
277 obj_val = model.obj()
278 return x_vals, obj_val, elapsed
279
280 #Print QIP results then QP results
281 for (subrace, attack), data in qp_problems.items():
282     Q, c = data["Q"], data["c"]
283
284     x_relaxed, obj_relaxed, t_relaxed = solve_relaxed_qp(Q, c)
285     x_integer, obj_integer, t_integer = solve_integer_qip(Q, c)
286
287     print(f"'Subrace':<10} {'Attack':<8} " + " ".join(f"{label:<6}" for label in stat_labels) + "
    ↪ Damage Time")
288
289     #QIP
290     print(f"{'':10} {'':8} " +
291           " ".join(f"{int(x):<6}" for x in x_integer) +
292           f" {obj_integer:<9.4f} {t_integer:<.3f}s")
293
294     #QP
295     print(f"{subrace:<10} {attack:<8} " +
296           " ".join(f"x:<6.1f}" for x in x_relaxed) +
297           f" {obj_relaxed:<9.4f} {t_relaxed:<.3f}s")
298
299     optimal_stats_qp[(subrace, attack)] = x_relaxed
300
301 #---- Applying Ability Multipliers----
302 def evaluate_damage(ability_mult, obj_func, toa_mult, x):
303     return ability_mult * obj_func(x) * toa_mult
304
305 #Dictionary for each subrace/attack type and the formula: Ability Multipliers, objective function,
    ↪ Attack type
306 damage_cases_qp = {
307     ("Vampire", "Melee"): (
308         0.765625,
309         lambda x: (1.06 + (0.03 / 250) * x[0] + (0.03 / 250) * (x[4]+50)) * (x[0] / 4 + 3 * x[5] / 80),
310         1
311     ),
312     ("Vampire", "Ranged"): (
313         0.765625,
314         lambda x: (1.06 + (0.03 / 250) * x[1] + (0.03 / 250) * (x[4]+50)) * (x[0] / 5 + x[1] / 20 + 3 *
    ↪ x[5] / 80),
315         1
316     ),
317     ("Vampire", "Magic"): (
318         0.65625,
319         lambda x: (1.16 + (0.12 / 250) * (x[2] + 50) + (0.04 / 250) * (x[4] + 50)) * ((x[2] + 50) / 4 +
    ↪ 3 * x[5] / 80),
320         0.75
321     ),
322     ("Vampire", "Spell"): (
323         1.25,

```

```

324         lambda x: (1.16 + (0.12 / 250) * (x[2] + 50) + (0.04 / 250) * (x[4] + 50)) * ((x[2] + 50) / 4 +
325             ↪ 3 * x[5] / 80),
326     2
327 ),
328 ("Werewolf", "Melee"): (
329     0.765625,
330     lambda x: (1.06 + (0.03 / 250) * (x[0] + 50) + (0.03 / 250) * x[4]) * ((x[0] + 50) / 4 + 3 *
331         ↪ x[5] / 80),
332     1
333 ),
334 ("Werewolf", "Ranged"): (
335     0.765625,
336     lambda x: (1.06 + (0.03 / 250) * (x[1] + 50) + (0.03 / 250) * x[4]) * ((x[0] + 50) / 5 + (x[1]
337         ↪ + 50) / 20 + 3 * x[5] / 80),
338     1
339 ),
340 ("Werewolf", "Magic"): (
341     0.65625,
342     lambda x: (1.16 + (0.12 / 250) * x[2] + (0.04 / 250) * x[4]) * (x[2] / 4 + 3 * x[5] / 80),
343     0.75
344 ),
345 ("Werewolf", "Spell"): (
346     1.375,
347     lambda x: (1.16 + (0.12 / 250) * x[2] + (0.04 / 250) * x[4]) * (x[2] / 4 + 3 * x[5] / 80),
348     2
349 ),
350 ("Werepyre", "Melee"): (
351     1,
352     lambda x: (1.06 + (0.03 / 250) * (x[0] + 35) + (0.03 / 250) * x[4]) * ((x[0] + 35) / 16 + (x[2]
353         ↪ + 35) / 16 + 3 * x[5] / 80),
354     1
355 ),
356 ("Werepyre", "Ranged"): (
357     1,
358     lambda x: (1.06 + (0.03 / 250) * x[1] + (0.03 / 250) * x[4]) * (x[1] / 8 + 3 * x[5] / 80),
359     1
360 ),
361 ("Werepyre", "Magic"): (
362     1,
363     lambda x: (1.16 + (0.12 / 250) * (x[2] + 35) + (0.04 / 250) * x[4]) * (3 * (x[0] + 35) / 40 + 3
364         ↪ * (x[2] + 35) / 40 + 5 * x[5] / 80),
365     0.9975
366 ),
367 ("Werepyre", "Spell"): (
368     1.5,
369     lambda x: (1.16 + (0.12 / 250) * (x[2] + 35) + (0.04 / 250) * x[4]) * (3 * (x[0] + 35) / 40 + 3
370         ↪ * (x[2] + 35) / 40 + 5 * x[5] / 80),
371     2
372 ),
373 }
374
375 print("")
376 print("Total Damage from QP Solution after Abilities")
377 for (race, atype), (ab_mult, formula, toa_mult) in damage_cases_qp.items():
378     x = optimal_stats_qp[(race, atype)]
379     dmg = evaluate_damage(ab_mult, formula, toa_mult, x)
380     print(f"{race:<10} {atype:<8} Damage: {dmg:.4f}")

```